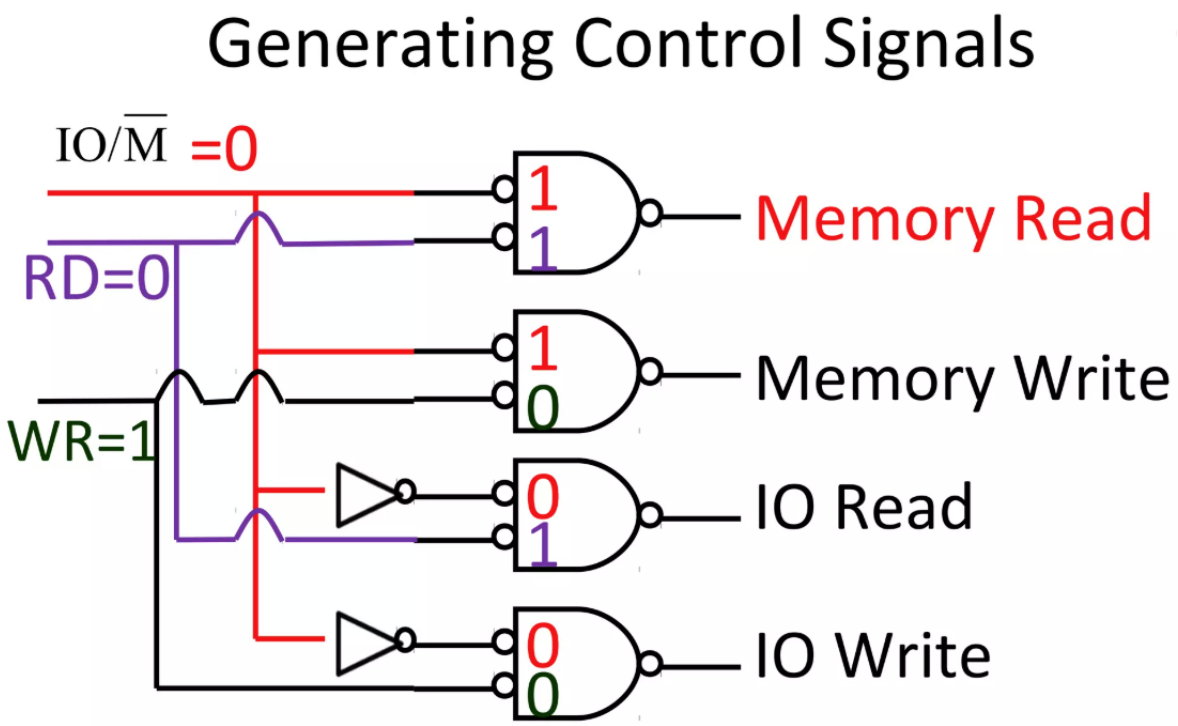
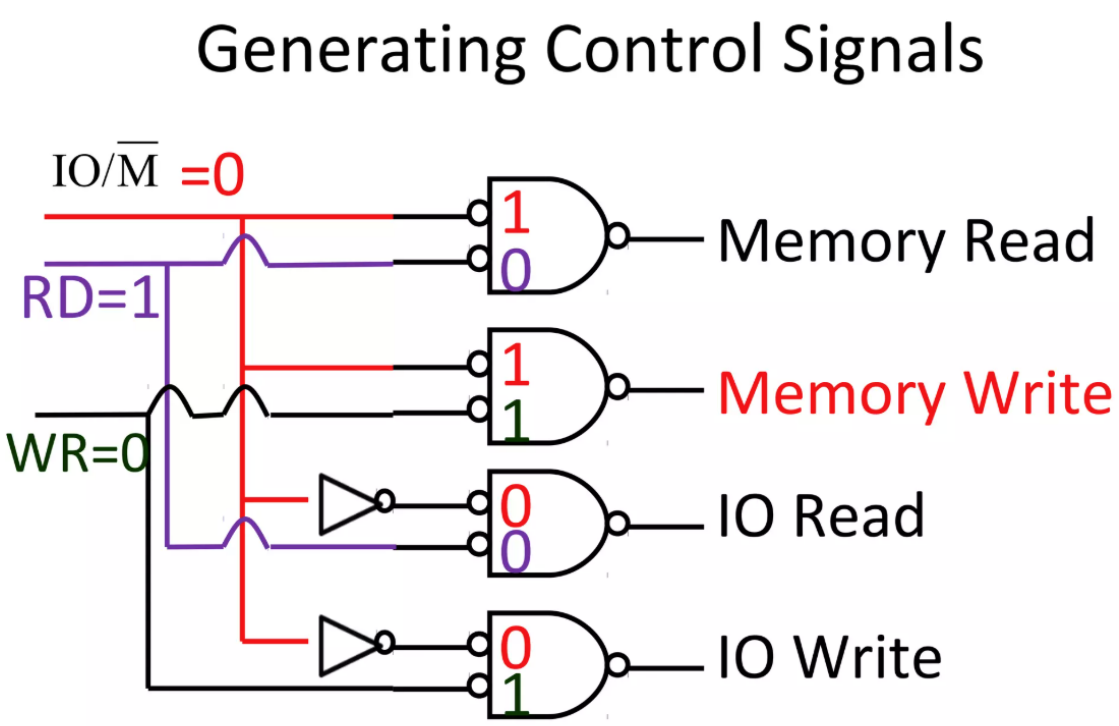
IRE 205: Microprocessor and Microcontroller System Design

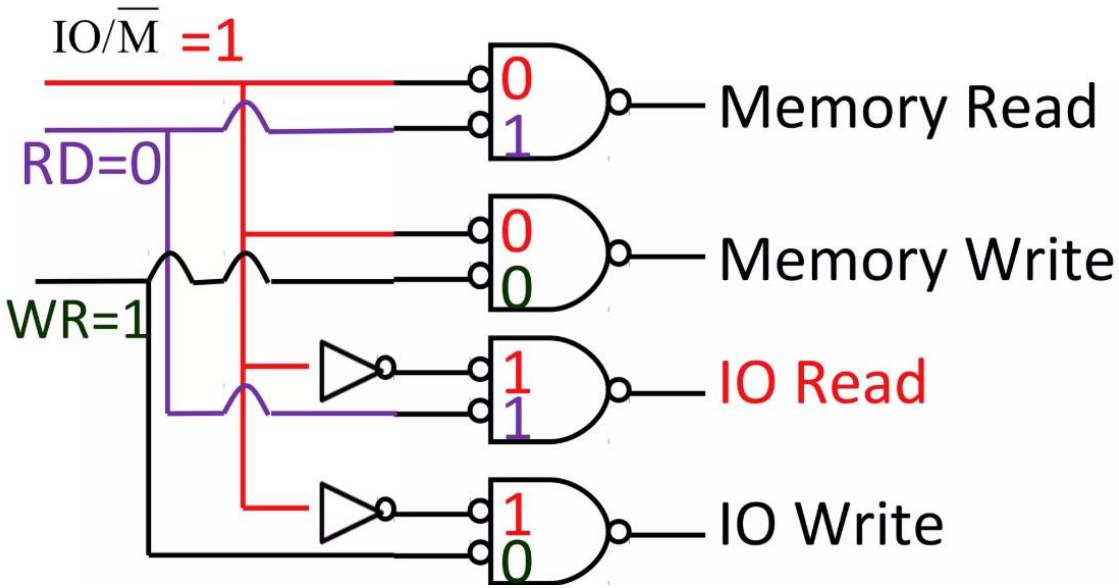
Introduction to Microprocessor

Control Signal for Microprocessor

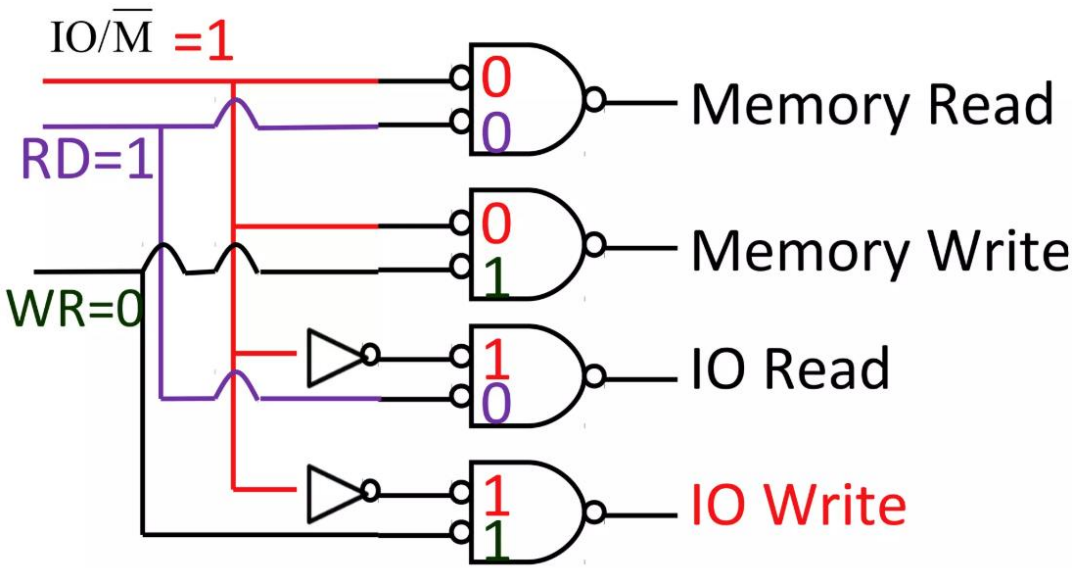




Generating Control Signals



Generating Control Signals



IO/M’	RD’	WR’	Status
0	0	1	Memory Read
0	1	0	Memory Write
1	0	1	IO Read
1	1	0	IO Write

Q1. Suppose, the processor wants to play a song. What will be the sequence of the control signal status to do so?

Answer: The processor will open the file and fetch data from memory→ Memory Read

Then it will send the data to a speaker → IO Write.

Q2. What will be the function of the buses while reading data?

Answer:

Address bus: Address buses put the address of memory location so that MUP can fetch data from it.

Control bus: Control buses will send the signal to instruct what actions to be done (Memory read).

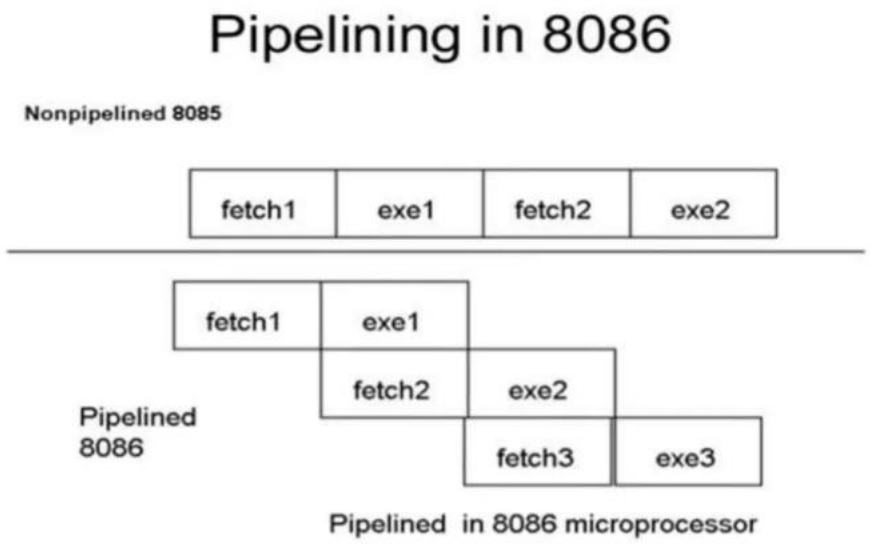
Data bus: The data stored on the location will be transmitted through data bus to MUP.

This will complete the process of fetching data.

Q3. What will be the function of the buses while writing the data?

Answer: ???

Pipelining



- Pipelining in 8086 means fetching and executing instructions at the same time.
- Instruction fetch and instruction execution overlap.
- 8086 uses an instruction queue to support pipelining.
- While one instruction is being executed, the next instruction is fetched.
- This reduces idle time of the processor.
- Pipelining increases execution speed.

Memory Banking

00000	34
00001	12
00002	
FFFFF	

1 MB memory chip

Even Bank

00000
00002
.
.
.
FFFFE

512 KB memory chip

Odd Bank

00001
00003
.
.
.
FFFFF

512 KB memory chip

- How 20 bit address lines can locate two consecutive addresses at time?

A19	A18	A17	A16	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	2
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	3
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	4
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.				
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.				
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	FFFFF

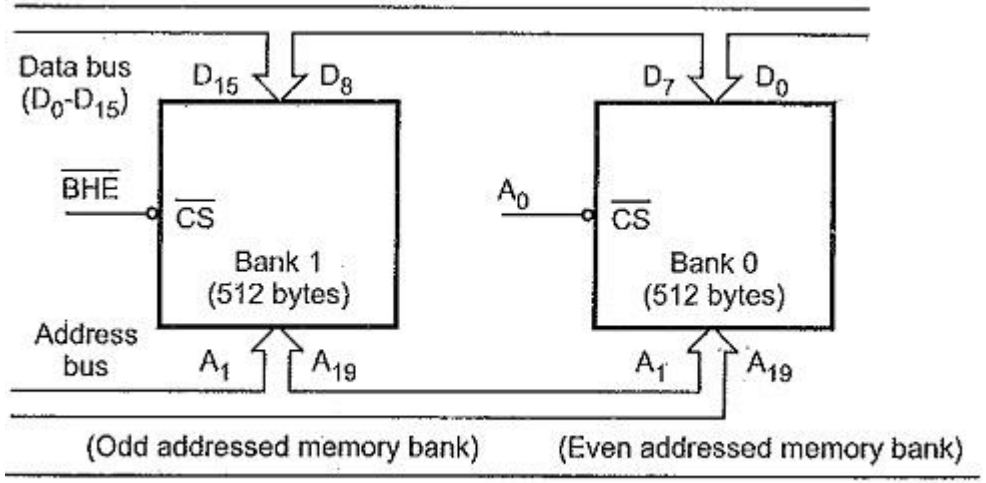


Fig. 10.11 Memory interfacing

No.	Operation	$\overline{BHE}$	A_0	Data Lines Used
1.	Read/Write a byte at an even address	1	0	D_7 - D_0
2.	Read/Write a byte at an odd address	0	1	D_{15} - D_8
3.	Read/Write a word	0	0	D_{15} - D_0

Architecture of 8086 MPU

8086 Microprocessor:

We will learn about Intel 8086 microprocessor as an example of CPU.

8086 is a 16 – bit microprocessor. That is - all the registers of 8086 is 16 - bit long. It has 20 – bit address bus. So it can access 2^{20} different memory locations. 2^{20} is called 1 MB. In other words, 8086 has 1 MB memory. So the problem is – the address is too big (20 – bit) to fit into the 16 – bit long register. This problem has been solved by segmented the memory logically.

Memory segmentation:

Memory of 8086 is logically segmented into four segments – i) Code Segment, ii) Data Segment, iii) Stack Segment and iv) Extra Segment.

Each memory segment is a block of 2^{16} (or 64 KB) consecutive memory bytes. Each segment is identified by a **segment number** (stored in the segment registers which is discussed later) starting with 0000h. As the segment number is 16 – bit long, so the highest segment number can be FFFFh.

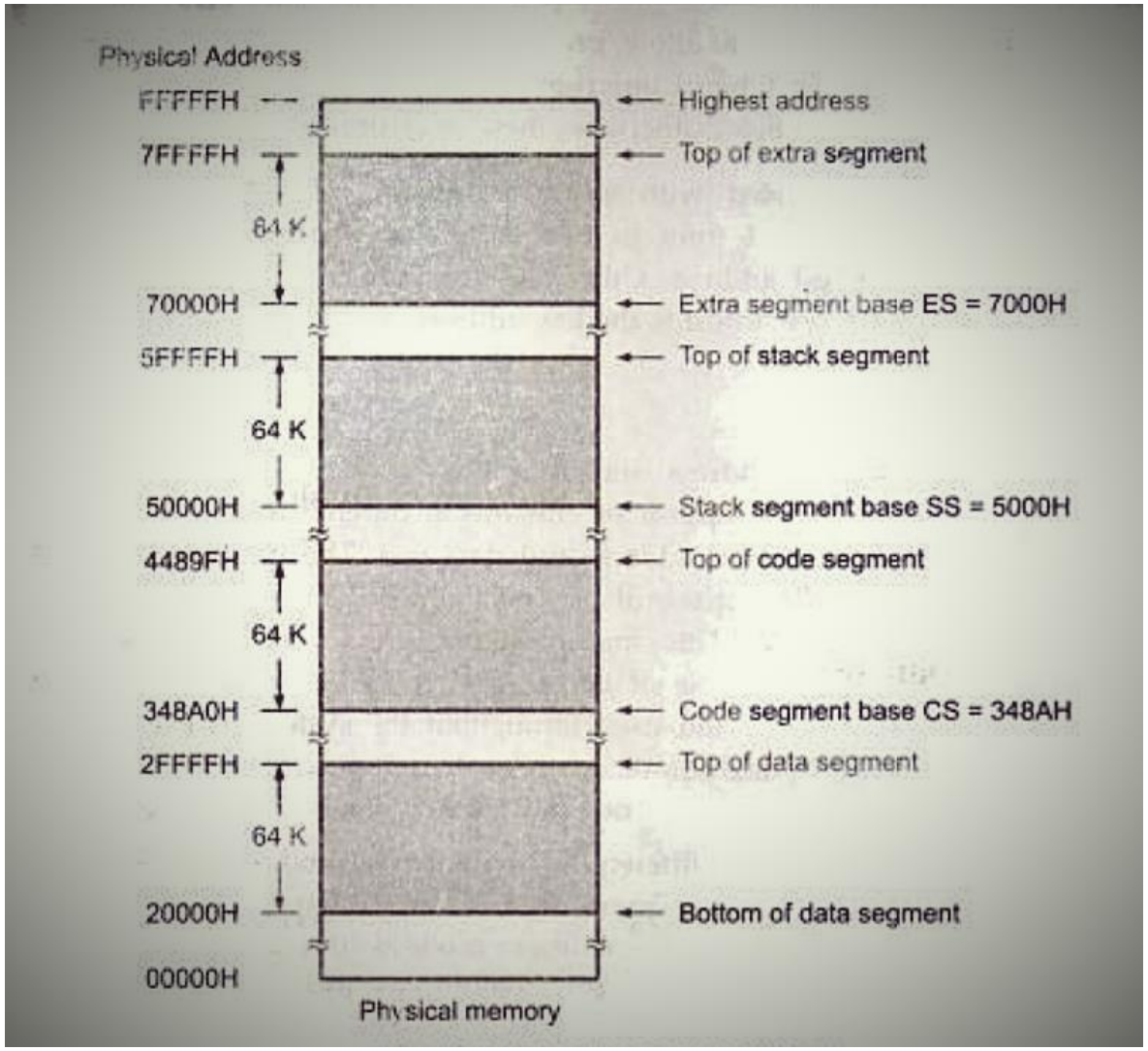
To access each memory location within the segment, an **offset** is given to them.

A 20 – bit physical address is written in the form Segment:Offset. This is known as **logical address**.

Logical address can be converted to physical address using the following formula:

$$\text{Physical Address} = \text{Segment} \times 10h + \text{Offset}.$$

Let the physical address of a memory location is 179B8h. If the start of the code segment is 1234h and the offset is 5678h, 179B8h can be presented by segment:offset.



Organization of 8086 MPU

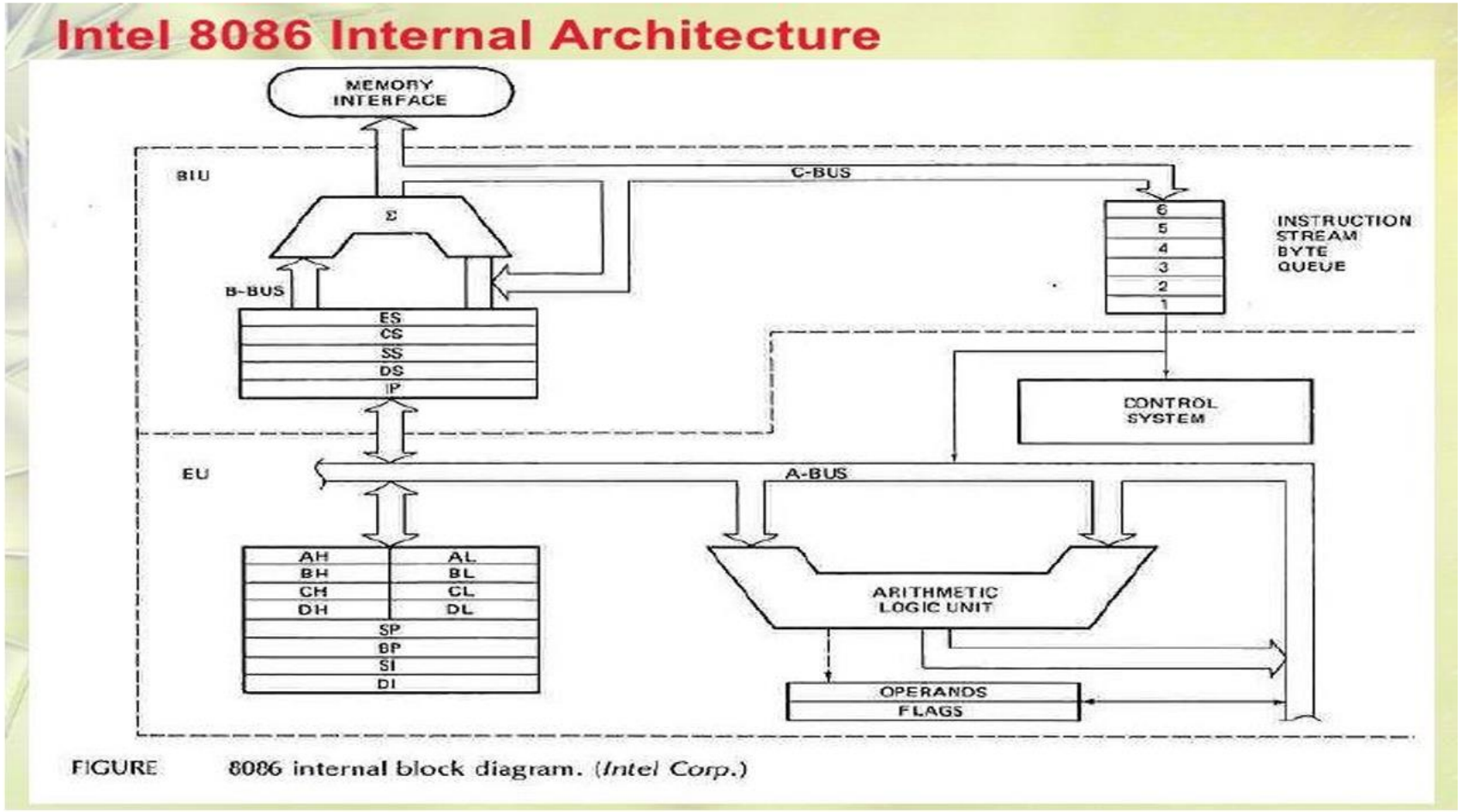


FIGURE 8086 internal block diagram. (*Intel Corp.*)

BIU (Bus Interface Unit):

It facilitates communication between the EU and the memory or I/O circuits. It handles all the data and addresses on the buses for EU. Bus operation includes instruction fetching, reading and writing operands for the memory and calculating the addresses of the memory location. BIU has four segment registers and one pointer register. They are – CS, DS, SS, ES and IP. Instruction queue is also a part of BIU. There is also a part in BIU called 'address compute engine' which converts the logical address into the physical address (denoted by 'Σ' in the following figure).

Instruction queue is an internal memory holding instruction. In 8086 microprocessor, it contains six consecutive instructions. It is included in BIU to speed up microprocessor. EU is much faster than peripheral part i.e. memory (RAM/ROM). That is why, architect in advance, stores specific instructions in the instruction queue that would be executed in future, while EU is executing the current instruction. This technique is called '**pipelining**'. So pipelining is a technique wherein BIU pre-fetches instructions for EU.

Segment Register:

Segment registers are used to address memory location and I/O space. We have seen that the memory of 8086 is logically segmented into four parts – Code Segment, Data Segment, Stack Segment and Extra Segment. Segment registers hold the starting addresses of the four memory segments. To access each and individual location in memory, we need offset registers. Index and Pointer registers are used to hold offset.

- **CS:** It contains the starting address of the code segment. This address plus the offset value contained in the IP indicates the address of an instruction to be fetched or executed.
- **DS:** It contains the starting address of the data segment. This address plus the offset value refers to a specific location in the data segment.
- **SS:** It contains the starting address of the stack segment. This address plus the offset value contained in the SP is used for stack operation.
- **ES:** It contains the starting address of the stack segment. It is used by some string operation. String instructions always use ES and DI registers to calculate the physical address for the destination.

Each segment register is 16 – bit long. That is, each can address 2^{16} = 65536 different memory location. 2^{16} is called 64 KB. So each segment is 64 KB long.

You are the programmer who will decide how to map the 1 MB into different segments. Here is an example:

Mapping of the 1 MB memory of 8086: (starting and ending address of the four segments are shown)

EU (Execution Unit):

EU executes instructions from the instruction queue. It decodes instruction fetched by the BIU. Then it generates the control signal and executes instruction. It has a circuit called arithmetic logic unit (ALU).

The ALU can perform arithmetic (+, -, ×, ÷) and logic (AND, OR, NOT) operations. The data for the arithmetic and logic operations are stored in registers. EU has eight registers for storing data. They are AX, BX, CX, DX, SI, DI, BP and SP. The high and low byte of the data registers can be accessed separately. The high byte of AX is called AH and the low byte is AL. Similarly, the high and low bytes of BX, CX and DX are BH and BL, CH and CL, DH and DL respectively. This arrangement provides more registers to use when the data are of byte – size.

- **AX (Accumulator Register):** It is used in all arithmetic and logical operation. In multiplication and division operations, one of the numbers involved must be in AX or AL.
- **BX (Base Register):** It is an address register. It points in a specific memory location in data segment.
- **CX (count register):** it is used as loop counter in program loop construction. In string operation, it also serves as counter.
- **DX (data register):** it is used in division and multiplication operation and also in I/O operation.
- SI, DI, SP and BP are the index and pointer registers. They contain the offset of the logical address of memory.
- **SI (Source Index):** it is used to point memory locations in the data segment.
- **DI (Destination Index):** it performs the same function as SI. It addresses in the extra segment.
- **SP (Stack Pointer):** it points to a specific memory location in stack segment.
- **BP (Base Pointer):** it is also used to access the stack segment. But it points to any data on the stack unlike SP. It can be used to access other segments also.

Thank You!